Optimal chaos control through reinforcement learning

Sabino Gadaleta[a] and Gerhard Dangelmayr[b]
Colorado State University, Department of Mathematics, Engineering E121, Ft. Collins, Colorado 80523

(Received 4 December 1998; accepted for publication 28 April 1999)

A general purpose chaos control algorithm based on reinforcement learning is introduced and applied to the stabilization of unstable periodic orbits in various chaotic systems and to the targeting problem. The algorithm does not require any information about the dynamical system nor about the location of periodic orbits. Numerical tests demonstrate good and fast performance under noisy and nonstationary conditions. © *1999 American Institute of Physics.* [S1054-1500(99)00703-X]

A general algorithm for chaos control is introduced and applied to the stabilization of unstable fixed points and periodic orbits. The technique is based on reinforcement learning combined with a vector quantization of the state space. No analytical description of the underlying dynamical system nor knowledge of the fixed points to be stabilized is required. Numerical tests for the logistic map, the Hénon map, a forced oscillator model for the heartbeat and a nine dimensional Lorenz system demonstrate good and fast performance of the algorithm even under noisy and nonstationary conditions. The method can handle both perturbations of a control parameter as well as perturbations in the form of proportional pulses applied to a state variable. In addition it is demonstrated that the same algorithm can also solve the targeting problem if the learning goals are suitably modified.

I. INTRODUCTION

Many physical systems are known to exhibit chaotic dynamics in certain parameter regimes, where they display a rich variety of different behaviors. In principle, a chaotic system has access to an unlimited number of states which are, however, unstable and are visited by the system in an unpredictable manner. In many cases of interest, system performance can be improved by controlling the dynamics such that it is constrained to one of these unstable states. For an efficient control, techniques are needed which on the one hand enforce the system to reside in this state through *small* perturbations and on the other hand allow, if desired, to switch from one state to another state. The first task is commonly referred to as the chaos control problem and the second task is known as the targeting problem. In the targeting problem one usually has also an optimality condition, i.e., the additional constraint of minimizing some energy functional such as time, average control action or combinations of both.

A key approach to chaos control is the stabilization of one of the infinitely many unstable periodic orbits (UPOs) which are embedded in a chaotic attractor. This problem has been first addressed by Ott, Grebogi, and Yorke (OGY)[1] and since then has stimulated a large number of papers in which different techniques have been developed and applied to experimental systems (see Ref. 2 for an extensive bibliography).

Concerning ''real world applications'' which are influenced by noisy and nonstationary environments and where no analytical description in terms of a map or vector field is available, an ideal, general purpose chaos controller should show the following features: (1) Does not require an analytical description of the system or previous knowledge about the location of UPOs or fixed points, (2) be robust against noise and nonstationarity, (3) allow the stabilization of any UPO, no matter of its order, (4) solve the targeting problem, and (5) offer the possibility of performance in real time.

To our knowledge, none of the methods proposed so far meet all these requirements simultaneously. The OGY method applies perturbations only when the system is close to the desired UPO and, moreover, requires knowledge of the linearized map describing the behavior near the fixed point. Besides the sometimes extensive calculations involved in approximating the linearized map and finding fixed point locations from a time series, this method is inadequate for nonstationary systems or the targeting problem. Recently Christini and Collins[3] proposed a modification of the OGY method, which approximates the fixed point location during control and is applicable in real-time. This method requires, however, initially large parameter perturbations and is limited to the stabilization of flip-saddle unstable periodic fixed points. Another approach to controlling chaos is provided by delayed feedback control methods.[4–6] Here an input is applied to the system which is proportional to the difference between the present state and an earlier one. With suitably chosen proportionality constants and time delay the system follows a periodic orbit. If no knowledge about the system exists, finding the right parameters can be problematic.[7]

In the last few years, artificial intelligence learning algorithms have been applied to the chaos control problem. The advantage of these algorithms is that they are data oriented and hence do not rely on an analytic description of the system. Weeks and Burgess[8] used a genetic algorithm to produce a neural network feedback controller which requires no previous knowledge about the system to be controlled. However, since genetic algorithms are generally slow, a long learning phase is necessary to produce a successful network

[a]Electronic mail: sabino@lagrange.math.colostate.edu
[b]Electronic mail: gerhard@lagrange.math.colostate.edu

which excludes real time performances in most cases of interest. Der and Herrmann[9,10] proposed methods using a Kohonen network or *Q*-learning to find optimal control strategies, however, in these approaches the location of the fixed point must be specified precisely.

In this paper we present a general control method based on reinforcement learning that meets all the requirements mentioned above. In particular, no specific assumptions about the system are made and no knowledge about its dimensionality or the location of the UPOs is required. The proposed method can be used to find optimal strategies whenever control is attempted through repeated parameter perturbations. This includes most discrete time feedback control methods. The control strategy is established through interaction between the controller and its environment and involves no computations of parameters or constants, assumes no *a priori* knowledge about the system and requires no construction of maps. The control policy is established through two alternative learning procedures known from the field of reinforcement learning: *Q*-learning and Sarsa learning.

An advantage of our method is its simplicity, which makes it easily implementable in comparison with other artifical intelligence based methods. Despite of its simplicity it performs, as will be demonstrated in this paper, well even under noisy or nonstationary conditions. Moreover, targeting as well as the control of higher order UPOs is possible by suitable modifications of the learning goals. We, therefore, suggest that the proposed method offers the possibility of an ''intelligent black-box controller'' that is well suited to control experimental systems.

In the next section we first introduce the chaos control problem and then give a brief overview of the general concepts of reinforcement learning. The proposed chaos control algorithm follows by introducing suitable learning goals. We summarize this algorithm in Sec. II C and apply it in Sec. III to the one-dimensional logistic map, the two-dimensional Hénon map, a continous system which serves as a model for an arrhythmic heartbeat and a nine-dimensional Lorenz system. In Sec. IV we discuss the results and describe future research suggested by our method.

II. CHAOS CONTROL THROUGH REINFORCEMENT LEARNING

A. The control problem

We consider a dynamical system that is posed in some phase space and is driven by a nonlinear vector field, possibly influenced by noise, however, neither the phase space nor the governing vector field is assumed to be known. Instead, we assume that one has access to discrete measurements $\mathbf{s}_n = \mathbf{s}(t_n)$ of the system at times t_n, $n=1,2,\ldots$. The measurement points $\mathbf{s}_n$ are determined by intersection of the trajectories of the original system with a low-dimensional manifold S, referred to as state space, which may correspond to a Poincaré section or to the recording of a Lorenz map. Thus the dynamics in S is determined by a map

$$\mathbf{s}_{n+1} = \mathbf{f}(\mathbf{s}_n, \mathbf{p}), \tag{1}$$

which is also assumed to be unknown. The only assumptions we make is that certain parameters $\mathbf{p}=(p_1,\ldots,p_m)$ on which the system depends can be varied in a neighborhood of some fixed $\mathbf{p}_0$ and that the system behaves chaotically at $\mathbf{p}_0$.

The goal of chaos control is to stabilize the system at one of its unstable periodic orbits (UPOs). In the discrete setting an UPO may have period one (unstable fixed point) or possess a higher period in which case it is also called a fixed point of higher order. Our main purpose is to stabilize an UPO with a prescribed period. The period is specified in advance as goal of the learning procedure, however, no knowledge about the location of UPOs is required.

To establish control we apply *small* system perturbations at the observation times t_n. We distinguish perturbations of a control parameter as suggested by OGY-related methods and direct perturbations of a state variable which was first introduced by Matías and Güémez.[11–14] In both cases the perturbation is described by a scalar quantity u which we restrict to take on discrete values, $u \in U = \{0, u_1, \ldots, u_m\}$. In the applications described in Sec. III we chose the maximum value of u such that it results in small system perturbations. One may consider large perturbation but has to be aware that this might result in the creation of spurious orbits as it is the fact with most control methods which apply external perturbations.

The control task consists in finding a control policy which associates with each state $\mathbf{s}_n$ a control $u \in U$ such that the specified goal is approached in a good way. As criterion for the quality of the policy we use the number of time steps needed until the goal is achieved if control was started from a randomly chosen initial state. We call this *iterations per episode*, λ. Formally our algorithm would find an optimal policy for the specified set of controls U only in the limit of infinitely many learning steps. We, therefore, attempt to find an approximative optimal policy. As criterion for such a policy we require that the number of iterations per episode is considerably less then the number of iterations per episode for the uncontrolled system, λ_u (note, that a chaotic system approaches every point on its attractor with probability one due to ergodicity).

In this work we use reinforcement learning techniques, specifically the temporal difference learning algorithms known as *Q*- and Sarsa-learning, to approximate the optimal control policy.

B. Reinforcement learning

Reinforcement learning (RL) methods offer a solution to the problem of learning from interaction of a decision maker, called agent, with a controlled environment to achieve a goal. At each discrete time step $n=1,2,\ldots$ the agent receives a representation of the environment's state $\mathbf{w}_n \in W$, where W is the (finite) set of all possible states. On the basis of $\mathbf{w}_n$ the agent selects a control (or action) $u_n \in U$, the set of all available actions. The control is selected according to a policy π, which is described by a probability distribution $\pi_n(\mathbf{w},u)$ of choosing $u_n = u$ if $\mathbf{w}_n = \mathbf{w}$. One time step later, as a consequence of the control u_n, the agent receives a numerical reward r_{n+1} and a new state $\mathbf{w}_{n+1}$. The goal is to maximize

the accumulated rewards. Reinforcement learning methods offer ways of improving the policy through observations of delayed rewards to find an optimal policy which associates with each state an optimal control such that rewards are maximized over time.

Problems with delayed reinforcements are well modeled as finite Markov decision processes (MDPs). A finite MDP consists of a finite set of states $\mathbf{w}\in W$ and controls $u\in U$, a reward function $R(\mathbf{w},u)$ specifying the expected instantaneous reward of the current state and action, and a state transition function $P^u(\mathbf{w}',\mathbf{w})$ denoting the probability of a transition from state $\mathbf{w}$ to $\mathbf{w}'$ using control u. The model is Markovian if the state transitions depend only on the current state and control.

In general one seeks to maximize the expectation value of the discounted return

$$\hat{r}_n=\sum_{k=0}^{\infty}\gamma^k r_{n+k+1},\quad 0\leqslant\gamma<1. \tag{2}$$

Most reinforcement learning algorithms are based on estimating the state-action value function

$$Q^{\pi}(\mathbf{w},u)=E_{\pi}\{\hat{r}_n|\mathbf{w}_n=\mathbf{w},u_n=u\}, \tag{3}$$

which gives the value of a state-action pair under a certain policy. An optimal state-action value function $Q^*(\mathbf{w},u)$ is defined for given $(\mathbf{w},u)$ as the maximum over the set of all policies π

$$Q^*(\mathbf{w},u)=\max_{\pi}Q^{\pi}(\mathbf{w},u). \tag{4}$$

Given an optimal state-action value function $Q^*(\mathbf{w},u)$, choosing in any state $\mathbf{w}$ the action u with associated maximal value Q^*

$$u(\mathbf{w})=\arg\max_{u\in U}Q^*(\mathbf{w},u), \tag{5}$$

leads to an optimal control strategy. Here $\arg\max_u$ denotes the value of u at which the expression that follows is maximized.

One can show[15] that Q^* satisfies the fixed point equation

$$Q^*(\mathbf{w},u)=R(\mathbf{w},u)+\gamma\sum_{\mathbf{w}'\in W}P^u(\mathbf{w}',\mathbf{w})\max_{u'}Q^*(\mathbf{w}',u'). \tag{6}$$

Denoting the operator on the right hand side (r.h.s.) of Eq. (6) by $H(Q)$, i.e., $Q^*=H(Q^*)$, it can be shown[15] that H is a contraction operator and, therefore, the iteration $Q_{n+1}=H(Q_n)$ converges to Q^*. For unknown $R(\mathbf{w},u)$ and $P^u(\mathbf{w}',w)$, H can not be determined, then the solution of $Q_{n+1}=H(Q_n)$ can be approximated using the stochastic Robbins Monroe algorithm[16]

$$Q_{n+1}=Q_n+\beta_n[H_n(Q_n)-Q_n]\equiv Q_n+\Delta Q_n, \tag{7}$$

where $H_n(Q_n)$ is an estimate of $H(Q)$ which is derived from observation of a realization of the Markov process. The $\{\beta_n\}$ are positive and have to satisfy $\Sigma\beta_n^2<\infty$, $\Sigma\beta_n=\infty$ for convergence. In temporal difference (TD) learning methods[17] the estimate of ΔQ_n is based on observations of the present state $(\mathbf{w}_n,u_n)$ and the next state $(\mathbf{w}_{n+1},u_{n+1})$ together with the immediate reward r_{n+1} which estimates $R(\mathbf{w}_n,u_n)$. Only the value of Q_n at $(\mathbf{w}_n,u_n)$ is updated, i.e., $\Delta Q_n(\mathbf{w},u)=0$ if $(\mathbf{w},u)\neq(\mathbf{w}_n,u_n)$. The update at $(\mathbf{w}_n,u_n)$ depends on the choice of estimate for the sum

$$\sum_{\mathbf{w}\in W}P^{u_n}(\mathbf{w},\mathbf{w}_n)\max_u Q^*(\mathbf{w},u). \tag{8}$$

Estimating Eq. (8) by $\max_u Q(\mathbf{w}_{n+1},u)$ leads to Watkins Q-learning[18]

$$\Delta Q_n(\mathbf{w}_n,u_n)=\beta_n[r_{n+1}+\gamma\max_u Q(\mathbf{w}_{n+1},u)-Q(\mathbf{w}_n,u_n)]. \tag{9}$$

This procedure represents an off-policy method: The Q-values are updated independent of the policy the controller uses to select controls.

Estimating Eq. (8) by $Q(\mathbf{w}_{n+1},u_{n+1})$ leads to the Sarsa-learning algorithm,[19,20] which represents an on-policy method

$$\Delta Q(\mathbf{w}_n,u_n)=\beta_n[r_{n+1}+\gamma Q(\mathbf{w}_{n+1},u_{n+1})-Q(\mathbf{w}_n,u_n)]. \tag{10}$$

In nonstationary environments it is convenient to set $\beta_n=\beta=\text{const.}$ with $0<\beta\leqslant 1$.[21] Complete convergence is then not guaranteed but the controller continues to receive rewards and is able to adjust to changing environments. In the applications in Sec. III we set $\beta=\gamma=0.9$.

For convergence to the optimal state-value function Q^* it is necessary that the whole state-action space is explored.[22] To ensure this, control actions are chosen from the corresponding Q values according to a specified policy which initially chooses actions stochastically and is slowly frozen into a deterministic policy. This can, for example, be achieved through ϵ-greedy policies π_{ϵ}. Here, an action which is different from the greedy action (the one with maximal estimated action value) is chosen with probability ϵ, where ϵ is initially one and is slowly frozen to zero

$$u^*=\arg\max_u Q(\mathbf{w},u), \tag{11}$$

$$\pi_{\epsilon}(\mathbf{w},u)=\begin{cases}1-\epsilon+\epsilon/|U| & \text{if } u=u^*\\ \epsilon/|U| & \text{if } u\neq u^*\end{cases}. \tag{12}$$

In Eq. (12), $|U|$ denotes the cardinality of the set U, i.e., the number of elements contained in U.

We call a policy greedy or deterministic, if exclusively the action u^* is chosen ($\epsilon\equiv 0$). An optimal state-action value function Q^* associates with each state $\mathbf{s}$ a control u such that by performing control actions according to a greedy policy from Q^* the goal is achieved in a minimum number of iterations from any initial state.

C. The control algorithm

We now introduce a chaos control algorithm which is based on the two TD-learning algorithms described before. Since the latter require formally a finite state Markov pro-

cess, a discretized version of the state space S is needed. If S is a one-dimensional interval this is simply achieved by covering S uniformly by small intervals of which the midpoints $\mathbf{w}^{(i)}$ serve as finite representations of S. If S is higher dimensional, a vector quantization is performed based on observations of $\mathbf{s}_n$ over a time window in which the system behaves chaotically. Specifically we employ the Neural-Gas vector quantization technique[23] that extracts a finite set of reference vectors $\mathbf{w} \in W = \{\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(N)}\}$ which represent the state space in their Voronoi cells. The advantage of this technique is that the distributions of the $\mathbf{w}^{(i)}$ is matched optimally to the invariant measure of the attractor.[24] Alternative techniques to obtain a finite representation of S can also be used. Which technique is used is not relevant for the control algorithm. (In the one-dimensional case the uniform distribution can usually be made fine enough to guarantee good performance of the algorithm.) In any case, the projection $\mathbf{W}$: $S \rightarrow W$ of S onto the representation space W is defined by

$$\mathbf{w}(s) = \arg\min_{\mathbf{w} \in W} ||\mathbf{w} - \mathbf{s}||. \tag{13}$$

Observations of the states $\mathbf{s}_n$ result in a sequence $\mathbf{w}_n = \mathbf{w}(\mathbf{s}_n) \in W$. The goal of the control is formulated in terms of this sequence. If a fixed point is to be stabilized, the criterion which tests if the goal was achieved is $\mathbf{w}_n = \mathbf{w}_{n+1}$ (one-step control). For a period two UPO it is $\mathbf{w}_n = \mathbf{w}_{n+2}$, $\mathbf{w}_n \neq \mathbf{w}_{n+1}$. In this case one formally may consider the Cartesian product $W \times W$ as a Markov state space and interpret $(\mathbf{w}_{n-1}, \mathbf{w}_n)$ as a single state. Practically, this simply corresponds to using two time steps for the update (two-step control). One proceeds analogously if higher order UPOs are to be stabilized. Note that no knowledge about the location of the fixed point or UPO is required. The control algorithm will first identify locations of UPOs and then approximate an optimal strategy for reaching them in an optimal way through updating of the associated state-action values $Q(\mathbf{w}_n, u_n)$. For targeting the goal is simply $\mathbf{w}_n = \mathbf{w}^*$, where $\mathbf{w}^*$ is the reference vector closest to the target state $\mathbf{s}^*$.

In the form presented here, the algorithm can only distinguish orbits based on their period. If more orbits of one period are present the algorithm is able to identify the different orbits of the same period, if ϵ is kept large, such that a desired orbit could be stabilized after identification by defining the corresponding weight vector as the goal location.

Concerning the control set U we choose a fixed set of $m+1$ real values $U = \{0, u_1, \ldots, u_m\}$, $u_i \in R$. In each state $\mathbf{w} \in W$ one of the controls $u \in U$ can be applied. Each such state-action has associated a value $Q(\mathbf{w}, u)$.

As a reward function we choose to punish unsuccessful controls and to give zero reward for successful controls

$$r_{n+1} = \begin{cases} 0 & \text{if goal achieved} \\ -1 & \text{otherwise} \end{cases}. \tag{14}$$

Given an optimal state-action value function $Q^*(\mathbf{w}, u)$ and a set of reference vectors W, the controlled dynamics in case of one-step control can be written as

$$\mathbf{w}_n = \mathbf{w}(\mathbf{s}_n), \quad u_n = \arg\max_{u \in U} Q^*(\mathbf{w}_n, u), \tag{15}$$

$$\mathbf{s}_{n+1} = \mathbf{f}(\mathbf{s}_n + u_n \Delta\mathbf{s}, \mathbf{p}_0 + u_n \Delta\mathbf{p}). \tag{16}$$

The arguments in $\mathbf{f}$ reflect the specific form of the control action. If only parameter perturbations are applied we set $\Delta\mathbf{s} = 0$, while $\Delta\mathbf{p}$ is the coordinate vector that selects the particular component of $\mathbf{p}$ chosen to be perturbed. Analogously, if proportional pulses are applied, $\Delta\mathbf{p} = 0$ and $\Delta\mathbf{s}$ is the coordinate vector of the perturbed component of $\mathbf{s}$. Clearly one may consider also combinations of these approaches or more general perturbations described by matrix operations as well as multiparameter control through the introduction of additional value functions, however, in this work we examine only the above two simplest types of control perturbations. We emphasize that only the iterated states $\mathbf{s}_n$ are used in the control algorithm, knowledge of $\mathbf{f}$ is not required. In numerical experiments as described in the next section the $\mathbf{s}_n$ are of course computed from a given $\mathbf{f}$, however, in ''real world applications'' they would be observed experimentally.

The optimal state-action value function is approximated using Q- and Sarsa-learning through interaction of the controller with the system. All possible controls u are tried and their values in terms of rewards are used as feedback to update the corresponding state-action values $Q(\mathbf{w}(\mathbf{s}), u)$. In the case studies of Sec. III it turned out that Q-learning typically performed better than Sarsa-learning.

We distinguish between online and offline control. In offline control, after termination of each episode the system state is reset to a new, randomly chosen initial value. This enables the controller to learn the globally optimal policy Q^*. In situations where one is interested in controlling a running system as quickly as possible one does not need a global optimal policy. In these situations we perform control online. In online control, the system is not reset to a new state after termination of an episode and Q-values are updated *on the fly* (a term introduced by Christini and Collins[25]), thus allowing real-time control.

Furthermore we distinguish between ϵ-greedy and deterministic learning. ϵ-greedy learning uses ϵ-greedy policies, while during deterministic learning exclusively the action with maximal Q-value is chosen. Note, that for deterministic policies, Q- and Sarsa learning are equivalent. The complete algorithm for one-step control is summarized in Fig. 1.

III. RESULTS

A. Logistic map

The first example we consider is the logistic map with added noise

$$x_{n+1} = p_0 \cdot x_n (1 - x_n) + \eta_n. \tag{17}$$

It exhibits chaotic behavior for $3.569\,456 < p_0 < 4$. The noise η_n added to the system is chosen uniformly from the set $(-\alpha/2, \alpha/2)$. To avoid divergence we restrict η_n such that $0 < x_{n+1} < 1$. The condition $x_{n+1} = x_n$ yields the nontrivial fixed point $x_f = 1 - 1/p_0$ ($x_f \approx 0.737$ for $p_0 = 3.8$).

We attempted to stabilize a fixed point through small local perturbations u of the control parameter p_0, where u could take on only three values, $u \in \{0, 0.1, -0.1\}$. Since the

Initialize $Q(\mathbf{w},u)$: $Q(\mathbf{w},u)=0\ \forall(\mathbf{w},u)$

REPEAT

Initialize $\mathbf{s}$ (only once for online control)

Find $\mathbf{w}=\mathbf{w}(\mathbf{s})$, $u \leftarrow Q(\mathbf{w},\cdot)$

REPEAT (per episode)

Apply u and observe $\mathbf{s}'$

Find $\mathbf{w}'=\mathbf{w}(\mathbf{s}')$, $u' \leftarrow Q(\mathbf{w}',\cdot)$

Determine reward:

$$r'=\begin{cases} 0 & \text{if goal achieved} \\ -1 & \text{otherwise} \end{cases}$$

Sarsa-learning:

$$\Delta Q(\mathbf{w},u)=\beta[r'+\gamma Q(\mathbf{w}',u')-Q(\mathbf{w},u)]$$

Q-learning:

$$\Delta Q(\mathbf{w},u)=\beta[r'+\gamma \max_{u'} Q(\mathbf{w}',u')-Q(\mathbf{w},u)]$$

$\mathbf{s} \leftarrow \mathbf{s}'$; $u \leftarrow u'$; $\mathbf{w} \leftarrow \mathbf{w}'$

UNTIL $r'=0$

UNTIL control terminated

FIG. 1. Summary of the proposed chaos control algorithm for one-step control based on either Sarsa or Q-learning. Both the online and offline versions are shown.

data are one-dimensional and confined between 0 and 1 we chose 101 points uniformly distributed between 0 and 1 as a set of reference points

$$w \in W=\{0, 0.01, 0.02,...,1\}. \tag{18}$$

As a criterion for reaching a fixed point the condition $w(x_n)=w(x_{n+1})$ was used.

Given an optimal state-action value function $Q^*(w,u)$ the controlled dynamics can be written as

$$x_{n+1}=(p_0+u_n)\cdot x_n(1-x_n)+\eta_n, \tag{19}$$

$$u_n=\arg\max_u Q^*(w(x_n),u). \tag{20}$$

First we approximated $Q^*(w,u)$ without added noise, $\alpha=0$, through ϵ-greedy offline Q- and Sarsa-learning. The results are shown in Fig. 2. Here ϵ was slowly decreased to zero over a period of 4000 episodes. The decreasing slope of the graph shows that the goal is reached more and more quickly in each new episode as ϵ is decreased and thus, demonstrates convergence of the policy. For the last 1000 episodes we set $\epsilon=0$. The slope of the graph above 4000 episodes is then a measure for λ, the average number of iterations to reach the goal with the current policy. Q-learning with $\lambda_Q=5.62$ performed slightly better than Sarsa-learning with $\lambda_S=5.70$. To compare λ for the controlled and the uncontrolled system, λ was averaged over 2000 episodes starting from random initial values and terminating an episode if $|x_n-x_f|<0.005$. Using the approximated control policies we obtained $\lambda_Q=5.61$ and $\lambda_S=5.87$, compared to $\lambda_u=151.34$ for the uncontrolled system.

Figure 3 shows the controlled dynamics for $\alpha=0$ using the policy Q^* approximated through Q-learning. Beginning in the initial state $x=0.1$ the system was randomly perturbed into a new state every 200 iterations. We see that control is re-established quickly after each system perturbation.

An advantage of the presented control algorithm is that it can be used in online control applications and in nonstationary environments. To demonstrate this, Fig. 4 shows the result of a control performed online. The system was set in a random initial state before the controller was turned on and the algorithm started to update the Q-values. Learning was performed deterministically using Q-learning. The parameter

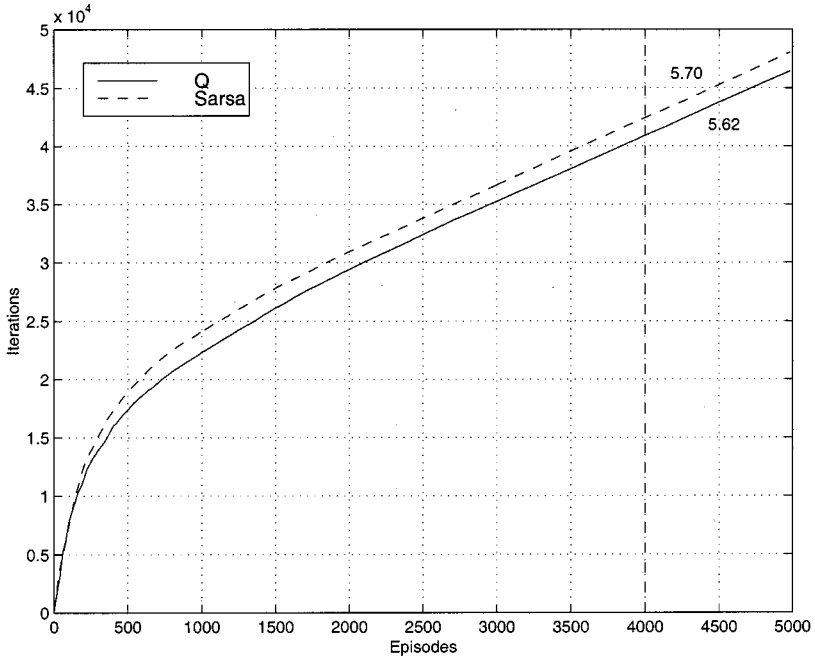

FIG. 2. The result of performing ϵ-greedy offline Q- and Sarsa-learning for the control of the logistic map with $\alpha=0$. The solid line shows the learning curve for the Q-learning algorithm, the dashed line for the Sarsa algorithm. ϵ was decreased to zero over the first 4000 episodes, after which $\epsilon=0$. The number close to the graph is the slope of the corresponding curve above 4000 episodes and is a measure of λ, the average number of iterations until the goal is reached when starting from a random initial condition and using the current control policy.

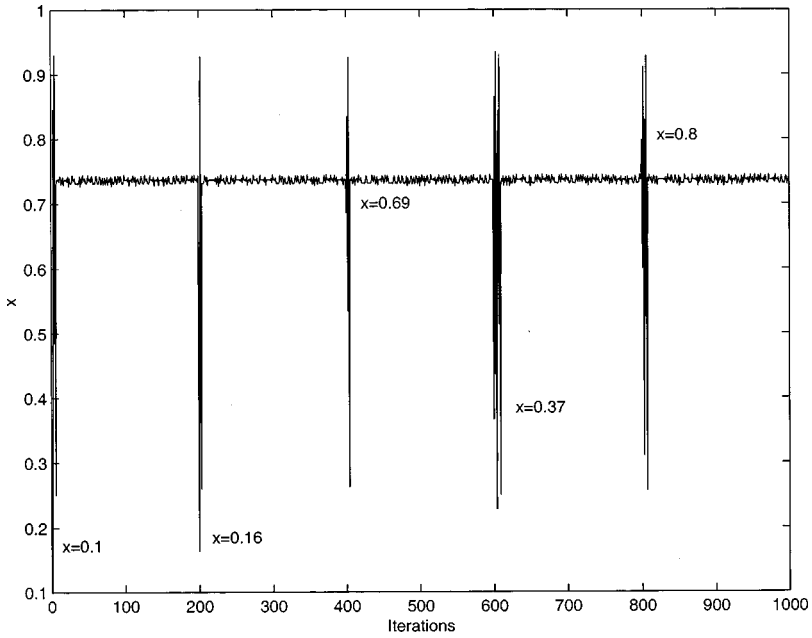

FIG. 3. The controlled dynamics of the logistic map with $\alpha=0$ using the policy Q^* obtained through Q-learning. Starting from a random initial state, the system is randomly kicked every 200 iterations into a new state as shown. Control of the nontrivial fixed point is re-established quickly after each perturbation.

p_0 was initially set to 3.89. After ~2500 iterations the controller establishes control of the fixed point x_f. After 10 000 iterations p_0 was set to 3.8. Control is lost for a short time, until the controller is able to relearn a control strategy and to stabilize the system at the changed fixed point location. After 20 000 iterations p_0 was set to 3.7 and finally after 30 000 iterations back to 3.89. Control is quickly reestablished in each case. This demonstrates that the proposed control algorithm is well suited for *on the fly* control and the control of nonstationary systems. Online performance is very fast since the controller only has to establish control starting from the particular initial value. We also see that ϵ-greedy learning is not necessary to establish control.

To demonstrate, that the control algorithm is able to detect and stabilize higher order fixed points, we attempted to stabilize fixed points of the logistic map up to period five through a deterministic online control. The result is shown in Fig. 5. Initially the goal was to stabilize a fixed point of period one. This was achieved after ~10 000 iterations of the logistic map. After 20 000 iterations, the goal was switched to control a period two fixed point, after 40 000 iterations to a period three and so on. In each case, locations of the fixed points were quickly identified and control established.

B. Analysis of the approximated policy

To get an idea of how the approximated control policy realizes control of a fixed point, we visualized the approximated policy Q^* for the logistic map in Fig. 6. The figure

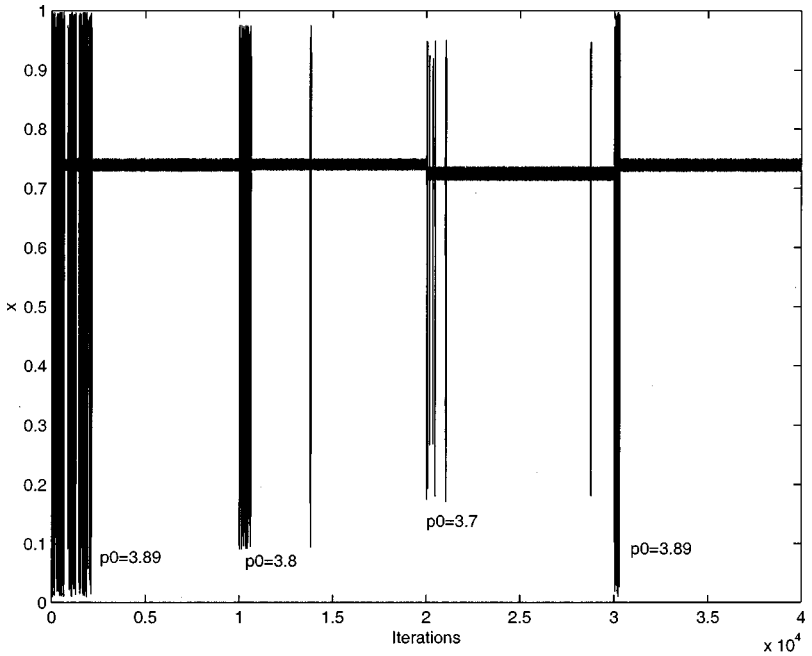

FIG. 4. Online control of a nonstationary logistic map. p_0 was changed every 10 000 iterations from initially 3.89 to 3.8, 3.7 and back to 3.89.

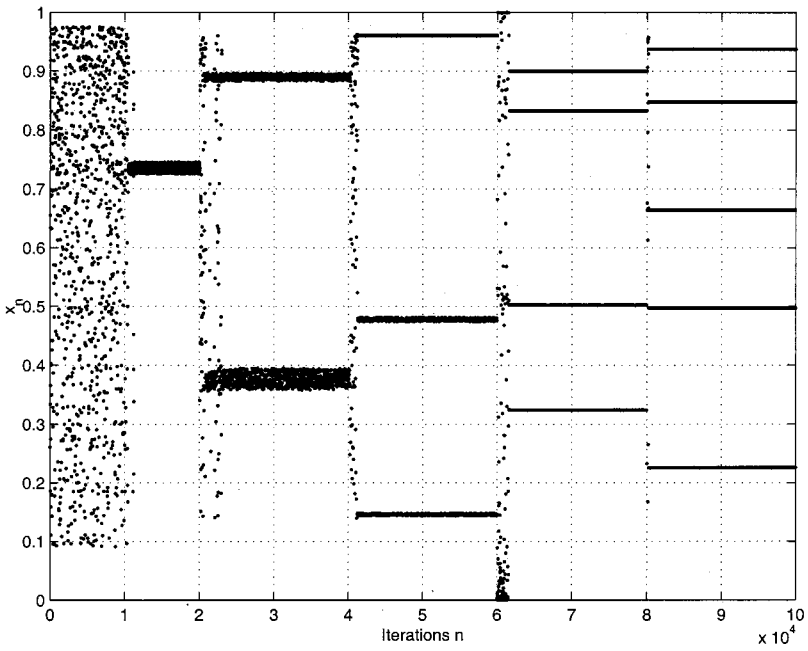

FIG. 5. Control of various fixed points of the logistic map through deterministic online control. The control goal was changed every 20 000 iterations.

shows for each reference vector the associated control with maximal Q-value. We then iterated the logistic map backwards from the fixed point x_f

$$x_n=(\tfrac{1}{2})(1\pm\sqrt{1-4x_{n+1}/p_0}). \tag{21}$$

The vertical lines in Fig. 6 correspond to the locations of backward iterated points. A vertical line labeled with integer i denotes a location from which the fixed point x_f can be reached in i iterations of the uncontrolled logistic map. Comparing the location of these lines and the controls u it becomes apparent that the controller learns to bring the dynamics globally close to these backward iterated locations, thus approximating a globally optimal policy. Close to x_f the controller performs a discrete OGY-control, while globally it brings the dynamics closer to points from which x_f can be reached in a minimum number of iterations.

C. Hénon map

The Hénon map

$$x_{n+1}=p_0+b_0x_{n-1}-x_n^2+\eta_n, \tag{22}$$

is a two-dimensional map for $\mathbf{x}_n=(x_n,x_{n-1})$ that behaves chaotically in a large range of the parameters b_0 and p_0.[26] Here η_n is again an added noise term, $\eta_n\in(-\alpha/2,\alpha/2)$. We choose $b_0=0.3$, then the map exhibits chaotic behavior in the range $1.1\leqslant p_0\leqslant 1.41$. When $b_0=0.3$, $p_0=1.29$, $\alpha=0$ the

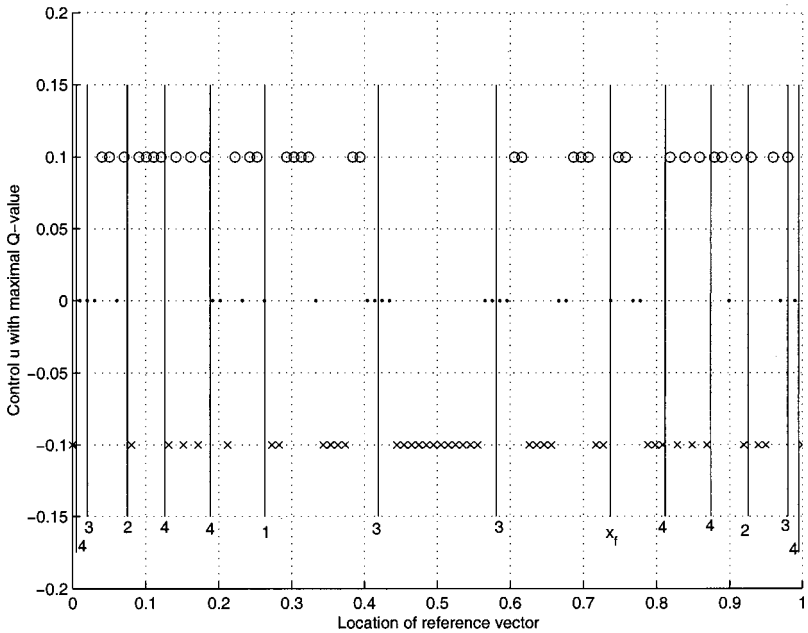

FIG. 6. The approximated policy Q^* (o for $u=0.1$, x for $u=-0.1$, . for $u=0$) as a function of the reference vectors. The vertical lines correspond to locations of points iterated backward from the fixed point x_f of the logistic map.

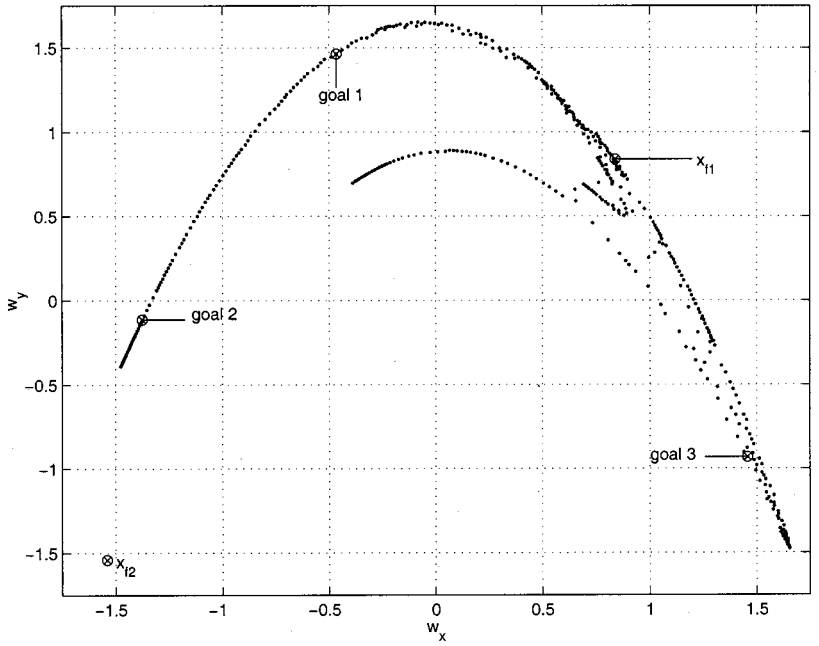

FIG. 7. The set of 430 reference vectors used for the control of the Hénon map. The locations of the fixed points $\mathbf{x}_{f1}$ and $\mathbf{x}_{f2}$ as well as the goals for the targeting application of Sec. III E are shown.

fixed points are $\mathbf{x}_{f1}\approx\{0.839, 0.839\}$ and $\mathbf{x}_{f2}\approx\{-1.539, -1.539\}$. We attempt to stabilize the system at a fixed point through small perturbations u of the control parameter p_0. The set of possible control actions consisted of three values, $u\in\{0,0.025,-0.025\}$. We can expect our control algorithm to stabilize the fixed point $\mathbf{x}_{f1}$ since the dynamics with small perturbations does not come close to $\mathbf{x}_{f2}$.

The used set of 430 reference vectors is shown in Fig. 7. The set was obtained through a vector quantization of 5000 datapoints ($p_0=1.29$, $b_0=0.3$, $\alpha=0$) using the Neural–Gas algorithm.[23,24]

Given an optimal state-action value function $Q^*(\mathbf{w},u)$ the controlled dynamics can be written as

$$x_{n+1}=p_0+u_n+0.3x_{n-1}-x_n^2+\eta_n, \tag{23}$$

$$u_n=\arg\max_u Q^*(\mathbf{w}(\mathbf{x}_n),u). \tag{24}$$

As for the logistic map, Q^* was approximated offline with ϵ-greedy Q- and Sarsa-learning. No noise was added and ϵ was slowly decreased to zero over a period of 9000 episodes. For the last 1000 episodes we set $\epsilon=0$. In Fig. 8 we see that Q-learning with $\lambda_Q=16.78$ performed slightly better then Sarsa-learning with $\lambda_S=17.56$. To compare λ for the controlled and the uncontrolled systems, λ was averaged over 2000 episodes starting from random initial values and

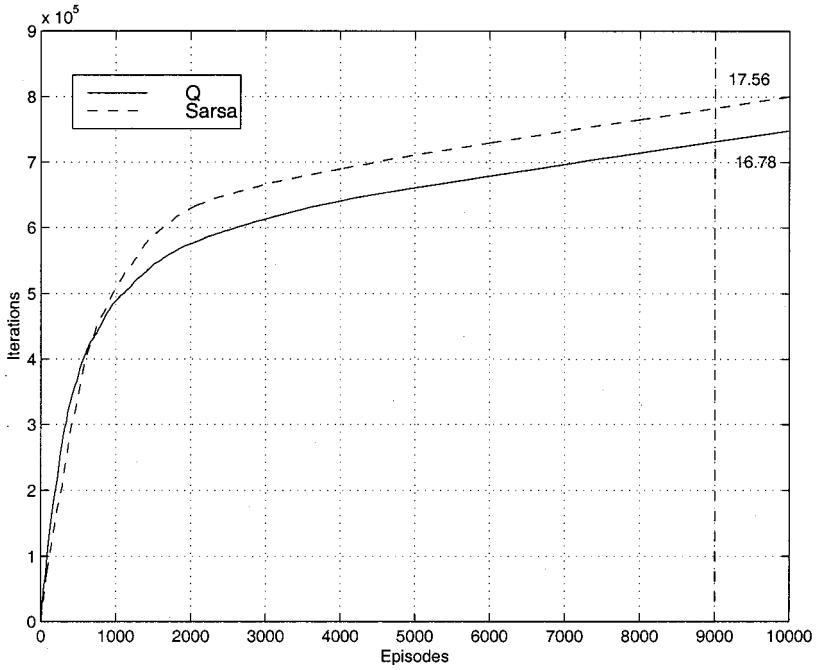

FIG. 8. The result of performing ϵ-greedy offline Q- and Sarsa-learning for the control of the Hénon map. The solid line shows the learning curve for the Q-learning algorithm, the dashed line for the Sarsa algorithm. ϵ was decreased to zero over the first 9000 episodes after which we set $\epsilon=0$. The slope of the graph after 9000 episodes is shown and a measure for λ.

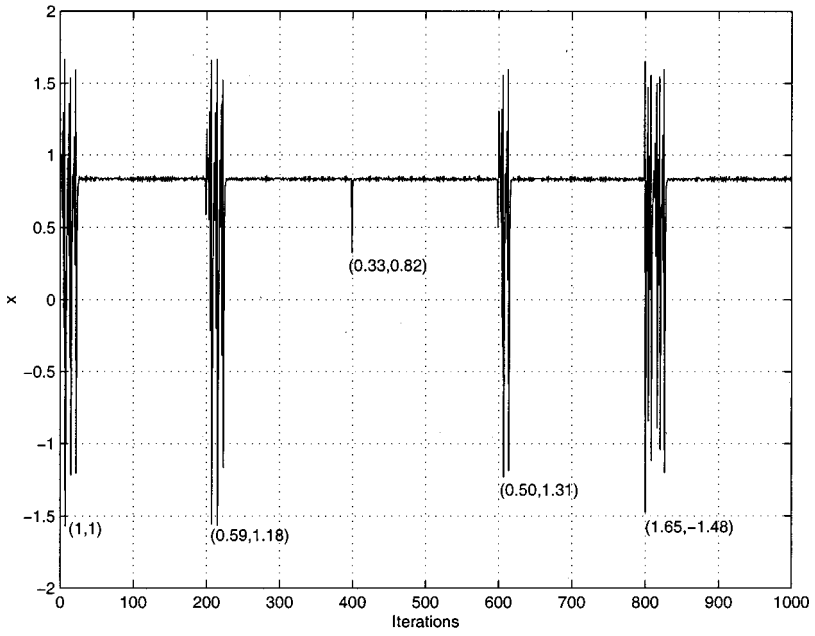

FIG. 9. Application of the Hénon control algorithm using the policy obtained via Q-learning. Starting from state (1,1) the system is randomly kicked every 200 iterations into a new state as shown. Control of the fixed point x_{f1} is re-established quickly after each perturbation.

terminating an episode if $\|\mathbf{x}_n-\mathbf{x}_f\|<0.02$. Using the approximated control policies we obtained $\lambda_Q=14.78$ and $\lambda_S=15.35$, while for the uncontrolled system $\lambda_u=1414$.

Figure 9 shows the controlled dynamics for the Hénon map using the policy Q^* obtained through Q-learning. Beginning in the initial state (1,1) the system was randomly perturbed into a new state every 200 steps as shown. We see that control is re-established quickly after each system perturbation.

The online control of a nonstationary Hénon map is shown in Fig. 10. The system was set in a random initial state before updating of the Q-values through the Q-learning algorithm was started. Learning was performed deterministic. p_0 was initially set to 1.33 and then changed after 20 000 iterations to $p_0=1.18$, after 35 000 iterations to 1.25 and then finally after 50 000 iterations to 1.29. We see that the controller is able to control this nonstationary system and adapt quickly to changes in the system parameter p_0. Note that for the whole control process the set of reference vectors shown in Fig. 7 was used. It was not necessary to introduce new centers or change center locations. This might, however, become necessary if the changed system visits a state space which is very different from the original one.

D. Noise dependence

To demonstrate resistance of the algorithm against noise, we compared λ_Q, the average number of iterations per episode with Q-control, to λ_u, the average number of iterations per episode without control. One episode was terminated

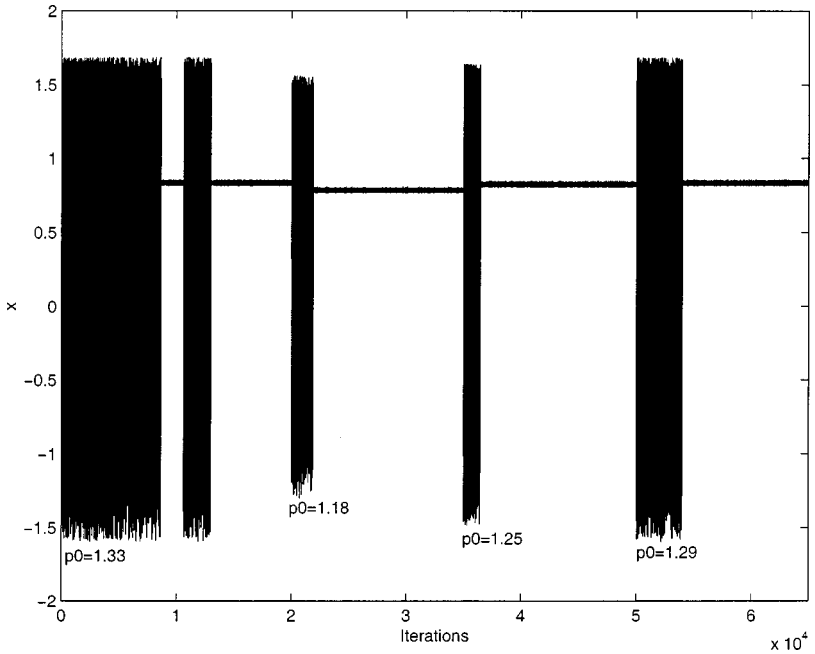

FIG. 10. Online control of a nonstationary Hénon map. p_0 was changed from initially 1.33 to 1.18, 1.25 and 1.29.

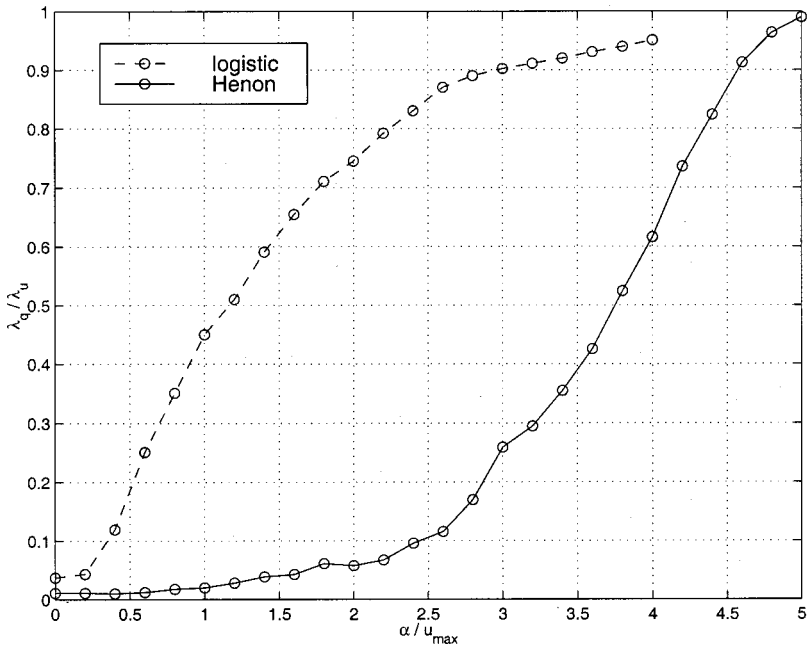

FIG. 11. Noise dependence of the algorithm. The dashed line gives λ_Q/λ_u as a function of $\alpha/u_{\max}$ for the logistic map, the solid line for the Hénon map.

when the system was close to the fixed point. For the logistic map we used $|x-x_f|<0.005$ and for the Hénon map $\|\mathbf{x}-\mathbf{x}_f\|<0.02$. After completion of one episode the system is set into a new random initial state. Figure 11 shows λ_Q/λ_u as a function of $\alpha/u_{\max}$. For the logistic map the maximal control $u_{\max}$ was 0.1, for the Hénon map 0.025. The policies were learned in a noisy environment with the given noise level. All policies were obtained offline through ϵ-greedy Q-learning. We see that the controller is able to reduce λ_Q below λ_u for considerable noise levels, even when the added noise is much higher then the applied control. It must be noted, however, that control of the fixed point is lost when $\alpha/u_{\max}\approx 1$ for the logistic map and $\alpha/u_{\max}\approx 2$ for the Hénon map. Nonetheless, for targeting applications, where we are only interested in reducing λ_Q as far as possible, the algorithm is resistant to high noise levels.

E. Targeting

To demonstrate that the algorithm can be applied to the more general problem of targeting we arbitrarily choose three different target locations for both the logistic map and the Hénon map. For the logistic map we choose $x_{g1}=0.1$, $x_{g2}=0.4$, and $x_{g3}=0.8$, for the Hénon map $\mathbf{x}_{g1}=(-0.47, 1.46)$, $\mathbf{x}_{g2}=(-1.38, -0.11)$, and $\mathbf{x}_{g3}=(1.46, -0.93)$ (see Fig. 7). No noise was added to the systems. We then learned optimal control policies using ϵ-greedy offline Q-learning presenting 10 000 episodes. An episode was terminated if the reference vector closest to the goal was reached. Using these learned control policies, we compared the average number of iterations needed to come close to the goal ($|x_n-x_{gi}|<0.005$ for the logistic map and $\|\mathbf{x}_n-\mathbf{x}_{gi}\|<0.02$ for the Hénon map) with applied control perturbations, λ_Q, and the average number of iterations until the uncontrolled system was close to the target location, λ_u. The applied controls and used sets of reference vectors were as described in the preceding sections. The results are shown in Table I. We see that λ is considerably reduced in all cases.

F. Relaxation oscillator and heartbeat

To demonstrate the control of continous systems from a time series we chose a system originally proposed by Rössler *et al.*,[27] who observed a variety of subharmonic responses for the periodically forced system

$$x' = (y-1.5)+(x-x^3/3)-A\sin(0.2t),$$
$$y' = -[x-0.467+0.8(y-1.5)]/9. \tag{25}$$

The subharmonic responses are similar to arrhythmias observed in malfunctioning hearts.[28–30] For $A=0.045$ the system shows chaotic behavior. The trajectory for $A=0.045$ in the (x,y)-plane is shown in Fig. 12.

TABLE I. Targeting of three different states for the logistic and the Hénon map.

	λ_Q	λ_u	λ_Q/λ_u
log. map, goal 1	7.9	∞	0
goal 2	12.6	178.6	0.07
goal 3	8.8	76.2	0.12
Hén. map, goal 1	30.0	498.9	0.06
goal 2	13.9	240.8	0.06
goal 3	195.6	1,142.9	0.17

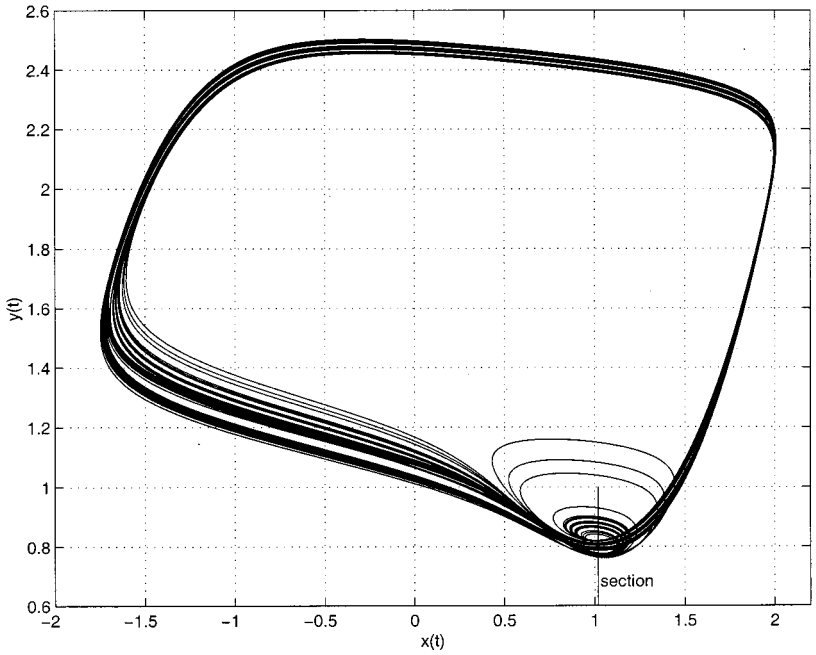

FIG. 12. Trajectory of system (25) in the (x,y)-plane. The experimental section used for the control is also shown.

To clear the system from the subharmonic responses we attempted to control the dynamics on a period one unstable periodic orbit through proportional pulses applied to the y-coordinate. As a criterion for successful control we chose that subsequent minimas of the y-coordinate have to occur at close values. Thus, the only information needed to control the system is contained in the time-series of the y-coordinate. To control the system we applied proportional pulses (or kicks) to the y-coordinate whenever y reached a local minimum

$$y \rightarrow u \cdot y \quad \text{if } y \text{ is at local minima.} \tag{26}$$

Control u was taken here from the set $\{0.9, 0.95, 1, 1.05, 1.1\}$. As reference vectors we chose the set of 41 points $W = \{0.6, 0.61, ..., 1\}$. In principle this control strategy represents a time discrete version of Pyragas' method. Instead of continously applying a feedback control to the variable $y(t)$ which brings the system close to a periodic orbit this feedback is only applied in the experimental section.

We applied online deterministic Q-learning to this control problem. After $\sim$140 visits of the section the controller was able to transform the arrhythmic signal into a periodic one. Figure 13 shows two parts of a plot of the signal $y(t)$ subject to the control perturbations. Figure 13(a) shows the signal for the first 50 visits of the section. In certain regions the signal becomes strongly perturbed while the controller evaluates possible actions. Figure 13(b) shows the signal just before the controller finally finds a successful strategy (after 140 visits of the section).

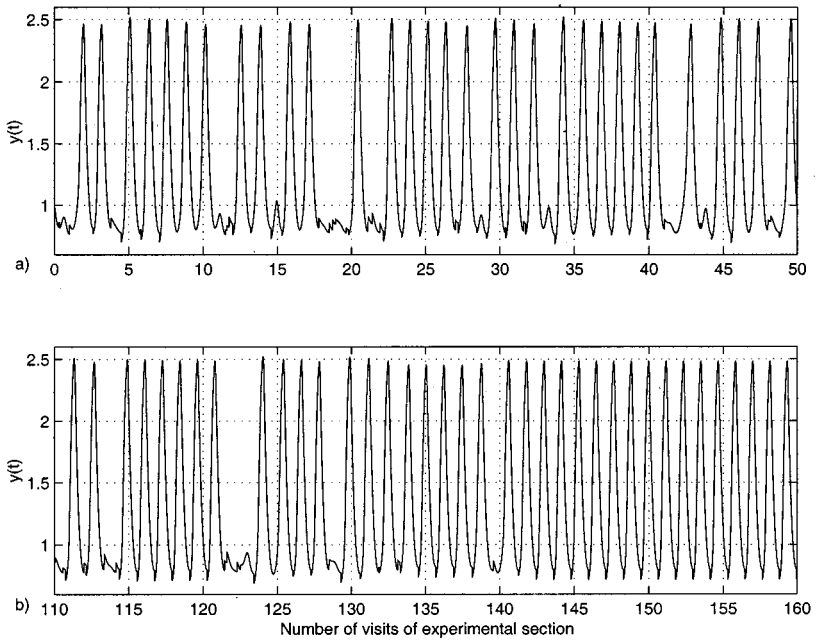

FIG. 13. The signal $y(t)$ subject to control perturbations while the Q-values are updated. (a) Shows the first part, (b) the last part, just before successful control was established.

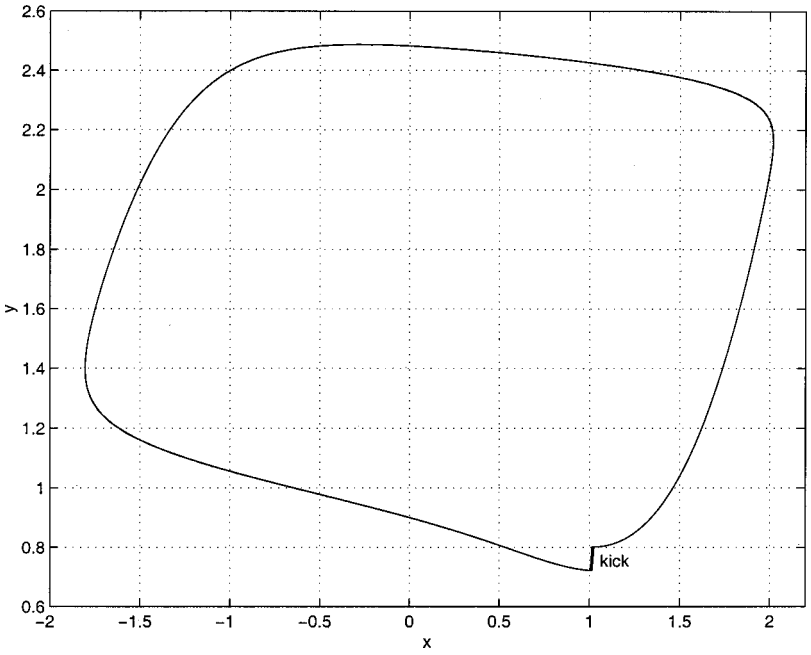

FIG. 14. Phase portrait of the controlled dynamics. The controller is able to clear the system from the subharmonic responses through kicking the y-coordinate whenever y reaches a local minimum.

The trajectory of the controlled system is shown in Fig. 14. When a local minimum in the y-coordinate is detected, y is perturbed from $y \approx 0.8$ to $y \approx 0.72$.

G. A system with two positive Lyapunov exponents

To investigate if the proposed control method can be applied to more complicated systems, in particular systems with two positive Lyapunov exponents, we attempted to control a nine-dimensional Lorenz system proposed by Reiterer *et al.*[31] They proposed the following system for the study of high-dimensional chaos:

$$\begin{aligned}
\dot{C}_1 &= -\sigma b_1 C_1 - C_2 C_4 + b_4 C_4^2 + b_3 C_3 C_5 - \sigma b_2 C_7,\\
\dot{C}_2 &= -\sigma C_2 + C_1 C_4 - C_2 C_5 + C_4 C_5 - \sigma C_9/2,\\
\dot{C}_3 &= -\sigma b_1 C_3 + C_2 C_4 - b_4 C_2^2 - b_3 C_1 C_5 + \sigma b_2 C_8,\\
\dot{C}_4 &= -\sigma C_4 - C_2 C_3 - C_2 C_5 + C_4 C_5 + \sigma C_9/2,\\
\dot{C}_5 &= -\sigma b_5 C_5 + C_2^2/2 - C_4^2/2,\\
\dot{C}_6 &= -b_6 C_6 + C_2 C_9 - C_4 C_9,\\
\dot{C}_7 &= -b_1 C_7 - r C_1 + 2 C_5 C_8 - C_4 C_9,\\
\dot{C}_8 &= -b_1 C_8 + r C_3 - 2 C_5 C_7 + C_2 C_9,\\
\dot{C}_9 &= -C_9 - r C_2 + r C_4 - 2 C_2 C_6 + 2 C_4 C_6 + C_4 C_7 \\
&\quad - C_2 C_8.
\end{aligned} \tag{27}$$

The b_i are defined in terms of a single parameter a as

$$\begin{aligned}
&b_1 := 4\frac{1+a^2}{1+2a^2}, \quad b_2 := \frac{1+2a^2}{2(1+a^2)}, \quad b_3 := 2\frac{1-a^2}{1+a^2},\\
&b_4 := \frac{a^2}{1+a^2}, \quad b_5 := \frac{8a^2}{1+2a^2}, \quad b_6 := \frac{4}{1+2a^2}.
\end{aligned} \tag{28}$$

Reiterer *et al.* set $a = \sigma = \frac{1}{2}$ and showed that the system exhibits a variety of different behaviors depending on the value of r. For $r > 43.3$ the system possesses two positive Lyapunov exponents which leads to hyperchaotic behavior. A projection of the dynamics generated by the system for the values $\sigma = a = 0.5$, $r = 44.0$ on the $C_1 - C_2$ plane is shown in Fig. 15.

By using our method we were able to control the system in the hyperchaotic regime, for $\sigma = a = 0.5$, $r = 44.0$, onto a periodic orbit of period one. Here perturbations have been applied to the complete set of state variables whenever the solution crosses the section defined by $C_1 = 0$, $\dot{C}_1 > 0$. A set W of 200 reference vectors $\mathbf{w}$ was constructed through a Neural–Gas vectorquantization of 1000 recorded crossings of the unperturbed system. Control was applied by using the method of periodic proportional pulses[14] in the section in the following way. If the orbit crosses the section in a state $\mathbf{C}$, C_2 is perturbed into the new state uC_2 with u chosen from $u \in U = \{1, 1.05, 1.1, -1.05, -1.1\}$, which brings the system into a state $\hat{\mathbf{C}}$. If u is different from one the complete state is set into the new state $\mathbf{C} = \mathbf{w}_n$, where $\mathbf{w}_n$ is the reference vector closest to $\hat{\mathbf{C}}$. We applied online deterministic Q-learning to this control problem. After ~ 100 visits of the section the controller was able to stabilize a period one dynamics. The stabilized orbit is shown in Fig. 16.

IV. CONCLUSIONS

In this paper we applied reinforcement learning to the problem of chaos control. The control strategy is based on learning state-action value functions for specific tasks. We considered in particular the task of stabilization of unstable fixed points or periodic orbits embedded in a chaotic attractor using small systems perturbations. The method does not require any analytic description of the underlying dynamical system nor do we need to know the location of the fixed

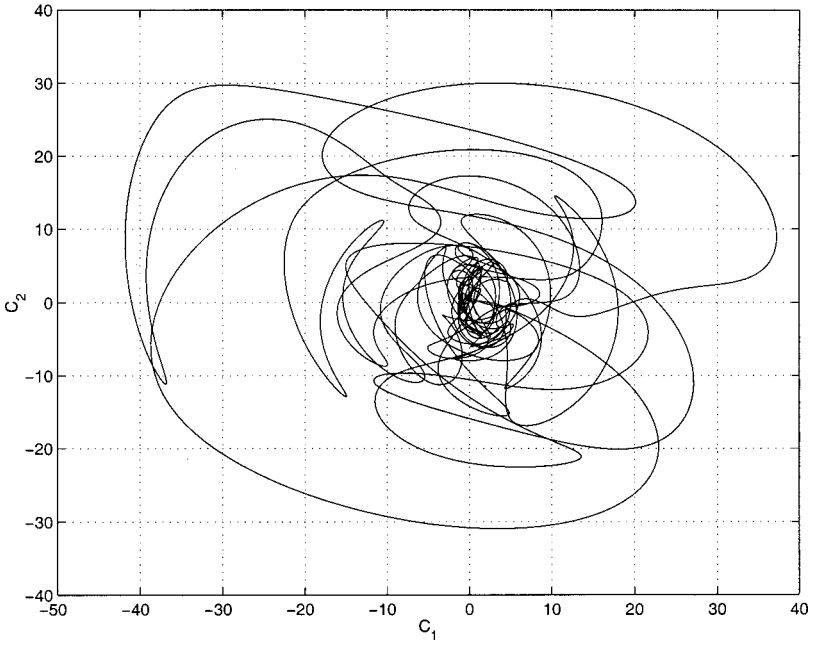

FIG. 15. Projection on the C_1-C_2 plane of the attractor generated by the 9D Lorenz system for $\sigma=a=0.5$ and $r=44.0$. The asymptotic behavior is characterized by two positive Lyapunov exponents.

points in advance. Roughly speaking, the algorithm first explores the reduced state space to determine an orbit that visits a neighborhood of the fixed point and once this neighborhood has been reached, a control policy is learned which represents locally an OGY-type method and traps the dynamics in the identified neighborhood while globally directing the dynamics as fast as possible into the identified neighborhood. Besides the stabilization of UPOs we also demonstrated for a simple case that the same algorithm, with suitably formulated goals, is capable of solving the targeting problem. Since it performs well under noisy as well as nonstationary conditions it offers promising perspectives for a general purpose black-box controller.

To satisfy the requirement of a finite Markov decision process, we first pursued a vector quantization to decompose the state space into a finite space, represented by a large number of Voronoi cells. The corresponding reference vectors have been determined by means of the Neural–Gas learning algorithm. Although this yields an optimal approximation of the invariant measure of the chaotic attractor, the resulting quantization of the state space may not be optimal for the specific control task under consideration. In future work we will, therefore, attempt to design a learning algorithm that performs the tasks of vector quantization and learning the optimal control strategy simultaneously. Another objective of future research will be to extend the algo-

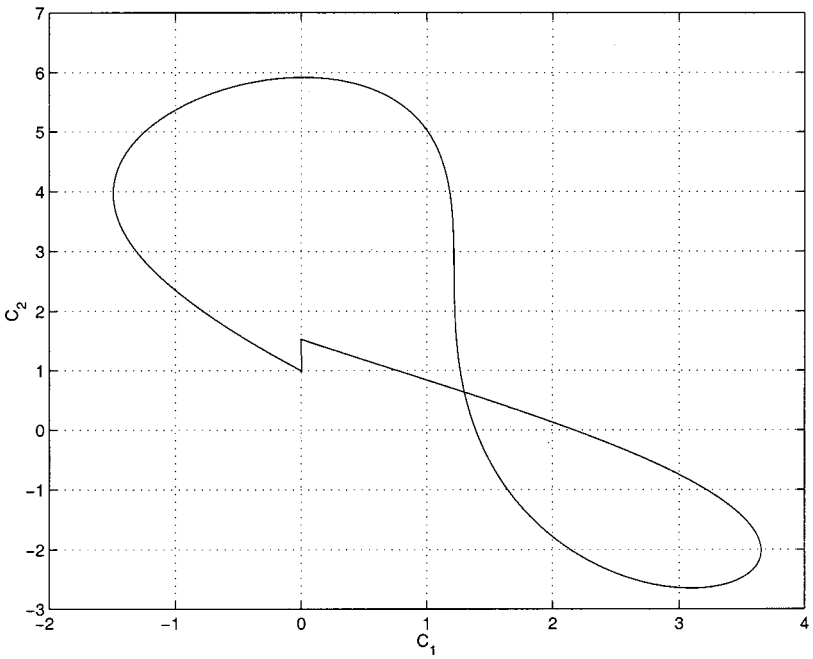

FIG. 16. Projection on the C_1-C_2 plane of the stabilized period one orbit.

rithm such that the targeting problem is solved subject to the constraint of minimizing a given cost functional.

Our results for the heartbeat model suggest that the control method could possibly be applied to atrial defibrillation. Atrial fibrillation is the most common sustained cardiac arrhythmia.[29] Recently first results with an implantable defibrillator restoring periodic rhythm through low-energy shocks were reported.[30] Since our algorithm performs well under nonstationary conditions, it could be well suited for designing ''intelligent'' implantable atrial defibrillators.

The results for the nine-dimensional Lorenz system in its hyperchaotic regime demonstrate the relevance of the presented control method to the control of more complex systems. Future research will concentrate on the question whether it is possible to control such complex systems through the perturbation of only a small subset of its state variables. We speculate that such investigation might lead to further insight into more general questions concerning chaos control in complex biological systems, in particular the relevance of chaos control for information processing in the brain.

ACKNOWLEDGMENTS

The authors wish to thank Dr. Charles W. Anderson for helpful discussions on reinforcement learning and the reviewers for their valuable comments.

[1] E. Ott, C. Grebogi, and J. Yorke, Phys. Rev. Lett. **64**, 1196 (1990).
[2] T. Kapitaniak, *Controlling Chaos* (Academic, San Diego, CA, 1996).
[3] D. Christini and J. Collins, IEEE Trans. Circuits Syst., I: Fundam. Theory Appl. **44**, 1027 (1997).
[4] K. Pyragas, Phys. Lett. A **170**, 421 (1992).
[5] K. Pyragas and A. Tamasevicius, Phys. Lett. A **180**, 99 (1993).
[6] M. Barrett, Physica D **91**, 340 (1996).
[7] A. Namanjunas, K. Pyragas, and A. Tamasevicius, Phys. Lett. A **204**, 255 (1995).
[8] E. Weeks and J. Burgess, Phys. Rev. E **56**, 1531 (1997).
[9] R. Der and M. Herrmann, *1994 IEEE International Conference on Neural Networks* (IEEE, New York, 1994), Vol. 4, pp. 2472–2475.
[10] M. Funke, M. Herrmann, and R. Der, Int. J. Adaptive Control and Signal Processing **2**, 489 (1997).
[11] M. Matías and J. Güémez, Phys. Rev. Lett. **72**, 1455 (1994).
[12] M. Matías and J. Güémez, Phys. Rev. E **54**, 198 (1996).
[13] N. Chau, Phys. Lett. A **234**, 193 (1997).
[14] N. Chau, Phys. Rev. E **57**, 378 (1998).
[15] S. Singh, Ph.D. thesis, University of Massachusetts, Department of Computer Science, 1993, also, CMPSCI Technical Report 93-77.
[16] H. Robbins and S. Monro, Ann. Math. Stat. **22**, 400 (1951).
[17] R. Sutton, Machine Learning **3**, 9 (1988).
[18] C. Watkins, Ph.D. thesis, University of Cambridge, England, 1989.
[19] G. Rummery and M. Niranjan, Technical report, Technical Report CUED/F-INFENG/TR 166, Engineering Department, Cambridge University (unpublished).
[20] R. Sutton, in *Advances in Neural Information Processing Systems: Proceedings of the 1995 Conference*, edited by D. Touretzky, M. Mozer, and M. Hasselmo, 1038 (1996).
[21] R. Sutton and A. Barto, *Reinforcement learning: An introduction* (Bradford, MIT, Cambridge, Massachusetts, 1998).
[22] L. Kaelbling, M. Littman, and A. Moore, J. Artificial Intelligence Research **4**, 237 (1996).
[23] T. Martinetz, S. Berkovich, and K. Schulten, IEEE Trans. Neural Netw. **4**, 558 (1993).
[24] T. Martinetz and K. Schulten, Neural Networks **7**, 507 (1994).
[25] D. Christini and J. Collins, Phys. Rev. E **53**, R49 (1996).
[26] M. Hénon, Commun. Math. Phys. **50**, 69 (1976).
[27] O. Rössler, R. Rössler, and H. Landahl, Sixth Int. Biophysics Congress, Kyoto, Abstracts 296 1978.
[28] J. Thompson and H. Stewart, *Nonlinear dynamics and Chaos* (Wiley, Chichester, Great Britain, 1986).
[29] W. Kennel, R. Abbott, D. Savage, and P. McNamara, N. Engl. J. Med. **306**, 1018 (1982).
[30] H. Wellens, C. Lau, B. Lüderitz, M. Akhtar, A. Waldo, J. Camm, C. Timmermans, H. Tse, W. Jung, L. Jordaens, and G. Ayers, Am. Heart J. **98**, 1651 (1998).
[31] P. Reiterer, C. Lainscsek, T. Schürrer, C. Letellier, and J. Maquet, J. Phys. A **31**, 7121–7139 (1998).